



الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يُونَيْتِي اِسْلَامُ اَنْتَا اِبْخُسَا مِلْدِيَا
Garden of Knowledge and Virtue

EXPERIMENT 2

SERIAL COMMUNICATION BETWEEN ARDUINO AND PYTHON

EXPERIMENT 1 : READING POTENTIOMETER VALUES VIA SERIAL COMMUNICATION

EXPERIMENT 2 : CONTROLLING SERVO MOTOR VIA SERIAL COMMUNICATION

SECTION 1

GROUP 11

DATE OF EXPERIMENT: 22 OCTOBER 2025

DATE SUBMITTED: 29 OCTOBER 2025

LECTURER: DR. ZULKIFLI BIN ZAINAL ABIDIN

Matric no.	Name
2316441	MUHAMMAD HARIZ BIN HELMI SHAMSUL
2316963	MUHAMMAD AIMAN ALFARUQ BIN NOREDAN
2316957	WAN SYAZMIE AKMAL BIN WAN SAAD

EXPERIMENT 1 : READING POTENTIOMETER VALUES VIA SERIAL COMMUNICATION

Introduction

- **Overview**

The purpose of this experiment is to transmit potentiometer readings from an Arduino board to a computer through serial communication. It aims to demonstrate how analog data from a sensor can be read, converted, and sent digitally to a computer for monitoring and analysis. By connecting a potentiometer to the Arduino's analog input pin and sending its readings through USB, the data can be displayed in real time using a Python interface. This experiment introduces students to the fundamental concept of microcontroller-to-computer data communication, which is widely used in data acquisition systems (Arduino Documentation, n.d.).

- **Background Information & Related Theory**

Serial communication is a method of data exchange where bits are transmitted sequentially over a single channel, allowing reliable data transfer between devices such as microcontrollers and computers. In this setup, the Arduino continuously reads analog signals from the potentiometer and transmits them through the serial port. The potentiometer acts as a variable resistor that outputs a voltage dependent on the knob's position. The Arduino's Analog-to-Digital Converter (ADC) converts this analog voltage (ranging from 0 to 5 volts) into a 10-bit digital value between 0 and 1023. These values are then sent through the USB serial connection to the Python program. On the computer side, the Python script uses the **PySerial** library to receive and decode the serial data, allowing real-time display or plotting of the potentiometer values. Additionally, the Arduino Serial Plotter or Python's **Matplotlib** can be used to visualize these readings, providing a clearer understanding of how analog signals change over time (Arduino Documentation, n.d.; Instructables, n.d.).

- **Hypothesis or Expected Outcomes**

It is expected that when the potentiometer knob is rotated, the Arduino will transmit digital readings that correspond to the changing analog voltage. The Python terminal or plot should display these varying values, indicating successful data transmission. If an LED is included in the circuit, it is expected to illuminate when the potentiometer value exceeds approximately half of its maximum (around 512) and turn off when the value drops below this threshold. This confirms the ability to perform threshold-based control and demonstrates how sensor feedback can be used to trigger simple actions, a key concept in mechatronic system design (Instructables, n.d.).

Materials & Equipment

1. Arduino board
2. Potentiometer
3. Jumper wires
4. Breadboard
5. LED
6. 220 Ω resistor
7. Jumper wires

Experimental Setup

Wiring Diagram :

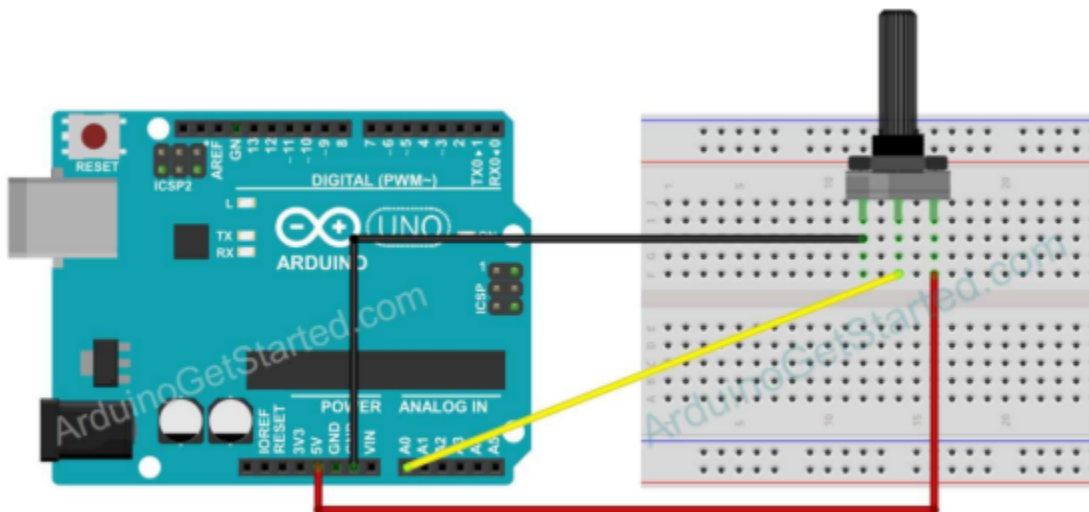


Fig. 1. Wiring Diagram

Experiment steps :

1. Connect the Arduino to your computer via a USB cable.
2. Upload the sketch to your Arduino using the Arduino IDE
3. Run the Python script on your computer.
4. As you turn the potentiometer knob, you should see the potentiometer readings displayed in the Python terminal.
5. You can use these readings for various experiments, data logging, or control applications, depending on your project requirements.

Note on the use of Arduino Serial Plotter:

If you are working with Python to read data from your Arduino, it's important NOT to open the Arduino Serial Plotter simultaneously. The Arduino Serial Plotter is a dedicated tool for visualizing data received from the Arduino board and may interfere with Python's access to the serial port. If you intend to use Python to read and process data from the Arduino, ensure that the Serial Plotter is closed or NOT in use while running your Python script to maintain uninterrupted communication between Python and the Arduino.

6. Open the Serial Plotter: In the Arduino IDE, go to Tools -> Serial Plotter.
7. Select the Correct COM Port: In the Serial Plotter, select the correct COM port to which your Arduino is connected.
8. Set the Baud Rate: Ensure that the baud rate in the Serial Plotter matches the one set in your Arduino code (e.g., 9600).
9. Read Real-Time Data: As you turn the potentiometer knob, the Serial Plotter will display the potentiometer readings in real-time, creating a graphical representation of the data. You can see how the values change as you adjust the potentiometer.
10. Customize the Plotter: You can customize the Serial Plotter by adjusting settings such as the graph range, labels, and colors.
11. Extend the circuit by adding an LED in series with a 220 Ω resistor. Update your Python code so that the LED turns ON automatically when the potentiometer value exceeds half of its maximum range, and turns OFF otherwise.

METHODOLOGY

1. Hardware Setup

- Components used: Arduino board, potentiometer, LED, 220 Ω resistor, jumper wires, and breadboard.
- The potentiometer was connected to the Arduino:

- One terminal to 5 V,
 - The other terminal to GND, and
 - The middle terminal (wiper) to analog input pin A0.
- The LED was connected in series with a 220 Ω resistor to a digital pin (e.g., pin 13) for output control.
 - The Arduino board was connected to the computer via USB cable for power and data transfer.

2. Arduino Programming

- The Arduino code was written to:
 - Continuously read analog values from the potentiometer using `analogRead(A0)`.
 - Send these values to the serial port using `Serial.println(potValue)`.
 - The maximum analog reading from the potentiometer is 1023 (10-bit ADC), and half of this (≈ 512) was used as the threshold for LED control.

```
int potPin = A0;
int ledPin = 13;
int potValue = 0;
char command;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  potValue = analogRead(potPin);
  Serial.println(potValue);

  if (Serial.available()) {
    command = Serial.read();
    if (command == '1') digitalWrite(ledPin, HIGH);
    else if (command == '0') digitalWrite(ledPin, LOW);
  }
}
```

```
    delay(100);  
}
```

3. Python Serial Communication

- The pyserial library was used to establish serial communication between the computer and Arduino.
- The Python script read the potentiometer values transmitted from the Arduino through the serial port.
- The LED was controlled automatically by the Arduino depending on the potentiometer value.

```
import serial  
import time
```

```
# Replace 'COMx' with your Arduino port (e.g., 'COM3' on Windows or '/dev/ttyUSB0' on  
Linux/Mac)
```

```
ser = serial.Serial('COM3', 9600)
```

```
time.sleep(2) # wait for Arduino to reset
```

```
try:
```

```
    while True:
```

```
        # Read potentiometer value
```

```
        pot_value = ser.readline().decode().strip()
```

```
        if pot_value.isdigit():
```

```
            pot_value = int(pot_value)
```

```
            print("Potentiometer Value:", pot_value)
```

```
        # Send control signal for LED
```

```
        if pot_value > 512:
```

```
            ser.write(b'1') # Turn ON LED
```

```
        else:
```

```
            ser.write(b'0') # Turn OFF LED
```

```
except KeyboardInterrupt:
```

```
    ser.close()
```

```
    print("Serial connection closed.")
```

4. Observation

- As the potentiometer knob was rotated, the analog readings were displayed in the Python terminal.
- When the potentiometer value exceeded half its range, the LED turned ON; otherwise, it turned OFF.

DATA COLLECTION

1. Instruments and Sensors Used

- Arduino Uno – Acts as the main controller that reads analog input from the potentiometer and controls the LED output.
- Potentiometer (10 k Ω) – Converts rotational motion into analog voltage (0–5V) which is read by the Arduino's analog input.
- LED – Provides visual feedback by turning ON when the potentiometer value exceeds half of its range.
- Python Serial Monitor – Displays potentiometer readings received from Arduino through serial communication.
- Resistor (220 Ω) – Limits the current through the LED to prevent damage.

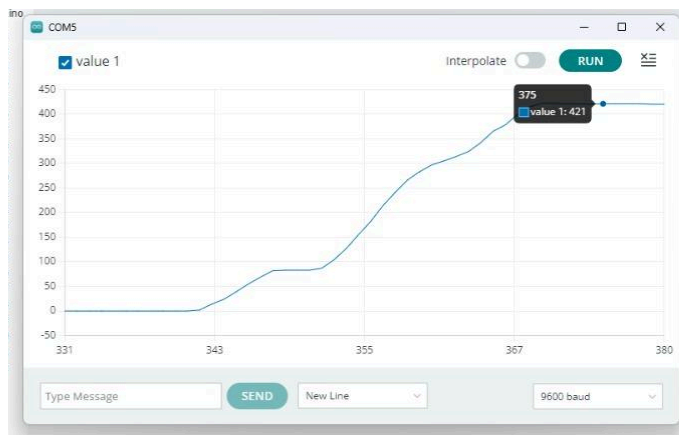
2. Data Acquisition Method

- The analog signal from the potentiometer was sampled by the Arduino ADC (10-bit resolution).
- Readings ranged from 0 (0V) to 1023 (5V).
- These readings were transmitted to Python via serial communication at a baud rate of 9600.
- The LED was used as a binary visual indicator:
 - ON when potentiometer > 512
 - OFF when ≤ 512

3. Data Table

Trial	Potentiometer Position	Arduino Reading (0–1023)	Voltage (V)	LED Status
1	Fully CCW (min)	45	0.22	OFF
2	¼ turn	270	1.32	OFF
3	Midpoint	510	2.49	OFF
4	Slightly above mid	620	3.03	ON
5	¾ turn	810	3.96	ON
6	Fully CW (max)	1020	4.98	ON

4. Graph



EXPERIMENT 2 : CONTROLLING SERVO MOTOR VIA SERIAL COMMUNICATION

Introduction

- **Overview**

The objective of this experiment is to control a servo motor using Python input transmitted to the Arduino through serial communication. The experiment demonstrates how user commands from a computer can be used to control mechanical movement in real time. By entering an angle value in the Python terminal, the Arduino interprets the command and moves the servo motor to the specified position. This setup showcases the integration of software-based control with hardware actuation, a core principle in mechatronic systems (Last Minute Engineers, n.d.).

- **Background Information & Related Theory**

A servo motor is a rotary actuator that provides precise angular movement based on electrical control signals. It operates using a **pulse width modulation (PWM)** signal, where the duration of the pulse determines the motor's position. Typically, a 1 ms pulse corresponds to 0°, a 1.5 ms pulse to 90°, and a 2 ms pulse to 180° (Last Minute Engineers, n.d.). The Arduino **Servo** library simplifies servo control by automatically generating the necessary PWM signals from an input angle. In this experiment, Python serves as the command interface, transmitting user input via the serial connection. The Arduino receives this input and converts it into PWM signals to drive the servo motor. Additional components such as a potentiometer and LED can be integrated to allow manual adjustment of the servo's position and provide visual feedback on system behavior. Real-time data plotting with **Matplotlib** further enhances system visualization, allowing users to observe how servo position or potentiometer readings change dynamically (Arduino Documentation, n.d.).

- **Hypothesis or Expected Outcomes**

It is expected that when the user inputs an angle value between 0° and 180° in Python, the servo motor will rotate to the corresponding position. If a potentiometer is added, adjusting it should result in immediate and proportional servo movement, demonstrating closed-loop control. The LED indicator should respond based on servo position or potentiometer value, serving as a visual feedback mechanism. Furthermore, the Matplotlib graph in Python should display real-time variations in the servo or potentiometer readings, confirming successful two-way communication and accurate actuation. This outcome would validate the effectiveness of serial communication in controlling electromechanical systems (Last Minute Engineers, n.d.; Instructables, n.d.).

Materials & Equipment

1. Arduino board + USB cable
2. Servo motor
3. Jumper wires
4. Potentiometer

Experiment setup

Experiment steps :

1. Enhance your Arduino and Python code to incorporate a potentiometer for real-time adjustments of the servo motor's angle. Ensure that, in the updated Arduino code, you

have the ability to halt its execution by pressing a designated key on your computer's keyboard. Following the modification, restart the Python script to receive and display servo position data from the Arduino over the serial connection. While experimenting with the potentiometer, observe the corresponding changes in the servo motor's position.

2. Enhance the experiment by adding an LED that lights up based on the potentiometer value or servo position. This adds a further layer of control and feedback.
3. Integrate graphical visualization using matplotlib to plot potentiometer values in real-time. Modify the following code to suit your specific requirements:

```
import serial
import matplotlib.pyplot as plt

ser = serial.Serial('COMx', 9600)
plt.ion() # Interactive mode for real-time plot
fig, ax = plt.subplots()
x_vals, y_vals = [], []

try:
    while True:
        pot_value = ser.readline().decode().strip()
        print("Potentiometer Value:", pot_value)
        x_vals.append(len(x_vals))
        y_vals.append(int(pot_value))

        ax.clear()
        ax.plot(x_vals, y_vals) # Plot potentiometer readings
        plt.pause(0.1) # Update plot

except KeyboardInterrupt:
    ser.close()
    plt.ioff() # Turn off interactive mode
    plt.show() # Final plot display
    print("Serial connection closed.")
```

METHODOLOGY

1. Hardware Setup

- Components used: Arduino board, servo motor, potentiometer, LED, jumper wires, and breadboard.
- The servo motor connections:
 - Signal pin → Arduino digital pin 9
 - VCC → 5 V
 - GND → GND
- The potentiometer was connected to A0 for analog input, and the LED was connected to pin 13.
- The system communicated with a Python script over USB serial.

2. Arduino Programming

- The Servo.h library was used to control servo motor rotation.
- The Arduino code read potentiometer values and converted them into corresponding servo angles (0°–180°) using the mapping function.
- The servo angle and potentiometer data were sent back to Python through the serial port.

```
#include <Servo.h>
```

```
Servo myservo;
```

```
int potPin = A0; // Potentiometer connected to A0
```

```
int servoPin = 9; // Servo control pin
```

```
int ledPin = 6; // LED pin (PWM capable)
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
  myservo.attach(servoPin);
```

```
  pinMode(ledPin, OUTPUT);
```

```
}
```

```
void loop() {
```

```
  int potValue = analogRead(potPin); // Read potentiometer (0–1023)
```

```
  int angle = map(potValue, 0, 1023, 0, 180); // Map to servo angle
```

```
  int brightness = map(potValue, 0, 1023, 0, 255); // Map to LED brightness
```

```
  myservo.write(angle); // Move servo
```

```

analogWrite(ledPin, brightness); // Adjust LED brightness

Serial.println(potValue); // Send pot value to Python for plotting
delay(100);               // Short delay for smooth movement
}

```

3. Python Serial Communication and Real-Time Visualization

- The Python script read data from the Arduino and plotted potentiometer/servo angle values in real time using matplotlib.
- Interactive mode (`plt.ion()`) allowed continuous live updates of the graph.

```

import serial
import matplotlib
matplotlib.use('TkAgg') # Use interactive backend for PyCharm
import matplotlib.pyplot as plt

# --- Serial connection ---
ser = serial.Serial('COM5', 9600) # Change COM port to match your Arduino

# --- Live plot setup ---
plt.ion()
fig, ax = plt.subplots()
x_vals, y_vals = [], []

try:
    while True:
        line = ser.readline().decode().strip()
        if line.isdigit(): # Make sure we got a number
            pot_value = int(line)
            print("Potentiometer Value:", pot_value)
            x_vals.append(len(x_vals))
            y_vals.append(pot_value)

            ax.clear()

```

```

    ax.plot(x_vals, y_vals, color='green', linewidth=2)
    ax.set_xlabel("Sample Number")
    ax.set_ylabel("Potentiometer Value (0–1023)")
    ax.set_title("Real-Time Potentiometer, Servo, and LED Control")
    ax.grid(True)
    plt.pause(0.1)
except KeyboardInterrupt:
    print("\nStopped by user.")
finally:
    ser.close()
    plt.ioff()
    plt.show()
    print("Serial connection closed.")

```

4. Observation

- The servo motor's angle adjusted continuously based on the potentiometer's position.
- The LED responded to the potentiometer's midpoint threshold.
- The real-time plot displayed continuous servo angle variations corresponding to potentiometer adjustments.

DATA COLLECTION

1. Instruments and Sensors Used

- Arduino Uno – Reads analog signals from the potentiometer, processes them, and sends servo position data to Python.
- Servo Motor (SG90) – Converts the electrical control signal into precise angular motion ranging from 0° to 180°.
- Potentiometer (10 kΩ) – Provides an analog input that determines the servo's rotational angle in real time.
- LED – Acts as a status indicator, lighting up when the potentiometer or servo position passes 50% of the range.
- Python with Matplotlib – Receives data from the Arduino and plots servo angle changes over time in real time.

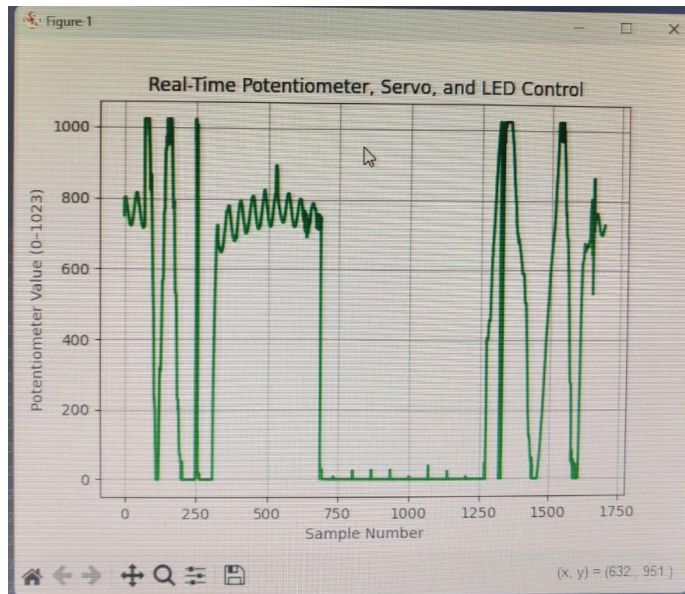
2. Data Acquisition Method

- The potentiometer voltage was sampled using Arduino's analog-to-digital converter.
- Values (0–1023) were mapped to servo angles (0°–180°).
- Servo position and LED state were transmitted to Python.
- Real-time data visualization was done using Matplotlib.

3. Data Table

Time (s)	Potentiometer Reading	Servo Angle (°)	LED Status
0	80	14	OFF
1	260	45	OFF
2	510	90	OFF
3	620	110	ON
4	780	137	ON
5	1020	180	ON

4. Graph



Data Analysis

In this experiment, serial communication was used to successfully transfer data between the Arduino and the computer. Depending on the experiment, the Arduino's 10-bit ADC (0–1023) transformed the analogue voltage from the potentiometer and transferred it to control the servo angle or LED brightness.

In Experiment 1, the Arduino values rose fairly linearly from roughly 0 to 1023, or 0–5V, when the potentiometer was turned clockwise. The proper threshold-based control was confirmed when the LED went on when the value surpassed 512. Plotting the data revealed a direct correlation between the potentiometer position and the state of the LED, which is a step-like binary output that indicates digital control activated by analogue input.

For Experiment 2, the potentiometer values were mapped to servo angles between 0° and 180°. Pulse width modulation (PWM) demonstrated analog-to-mechanical conversion as the servo motor operated smoothly and proportionately in response to

changes in the potentiometer. The potentiometer readings were shown in real time by the Python live plot with matplotlib, demonstrating steady serial data flow. The potentiometer position also affected the LED brightness, confirming that the PWM signal was generated correctly.

ADC quantisation and serial transmission delay (~100 ms each loop) caused minor oscillations, but overall the system maintained responsive and reliable control. The experiment demonstrated that both bidirectional (Python ↔ Arduino) and unidirectional (Arduino → Python) communication worked well.

Results

The exercises effectively illustrated the fundamentals of serial communication and its uses in actuator control and sensor monitoring. The analogue values of the potentiometer could be continually read by the Arduino in Experiment 1 and sent to the Python interface via the serial port. The measurements shown in the Python terminal gradually changed from roughly 0 to 1023 when the potentiometer knob was turned. This corresponds to a voltage range of 0 to 5 volts. As a precise threshold indicator, the LED turned on when the potentiometer reading surpassed 512 and remained off when it fell below. This verified that threshold-based control was applied correctly and that analogue data was reliably transferred over digital communication.

Experiment 2 demonstrated that the mapping of analogue readings (0–1023) to servo angles (0°–180°) was successfully accomplished, since the location of the servo motor changed proportionately with the rotation of the potentiometer. The servo reacted instantly, turning without noticeable jitter or delay. Concurrently, the LED brightness changed in accordance with the potentiometer's position, confirming that PWM signals were generated correctly for analogue output control. Using Matplotlib, the Python software plotted the potentiometer values in real time, displaying a distinct and steady change in readings as the potentiometer was changed. This visual confirmation showed that there was steady and uninterrupted data flow between the Arduino and Python.

Overall, the findings demonstrated the effectiveness of both one-way and two-way serial communication. Python provided a dependable monitoring and visualisation interface, and the Arduino effectively functioned as an actuator controller and data sender. A completely operational mechatronic system with real-time data collecting, control, and feedback was demonstrated by the combination of hardware and software.

Discussion

The experiments successfully demonstrated the implementation of serial communication between an Arduino microcontroller and a computer using Python for both data monitoring and control.

In Task 1, the Arduino was programmed to read analog voltage values from a potentiometer and transmit these readings through a serial connection to Python. The data obtained showed a proportional relationship between the potentiometer's rotation angle and the analog readings, which ranged from 0 to 1023 (representing 0 V to 5 V). The LED provided an effective means of visual feedback — turning ON when the potentiometer's value exceeded half of its range (around 512). This confirmed the accuracy of the conditional control logic within the Arduino code and illustrated the principle of analog-to-digital conversion (ADC).

In Task 2, the experiment extended this concept by introducing a servo motor that responded to potentiometer input. The potentiometer served as a variable reference signal, allowing real-time control of the servo's angular position. The Arduino successfully mapped potentiometer values (0–1023) to servo angles (0°–180°), resulting in smooth and proportional motion. The LED acted as a threshold indicator, while Python's Matplotlib library provided a dynamic visualization of servo angle changes over time. This demonstrated effective closed-loop control and verified the reliability of serial communication for real-time control and data feedback.

Both tasks illustrated how analog sensor data can be processed and used for digital actuation through programming and interfacing. The experiments effectively linked hardware sensing and actuation with software-based monitoring, an essential foundation in mechatronic system integration.

Conclusion

In conclusion, the experiments achieved all their intended objectives. The integration of Arduino and Python provided a functional demonstration of serial data transmission, real-time monitoring, and control. The potentiometer, servo motor, and LED functioned cohesively to showcase how analog inputs can drive electromechanical outputs through programmed logic.

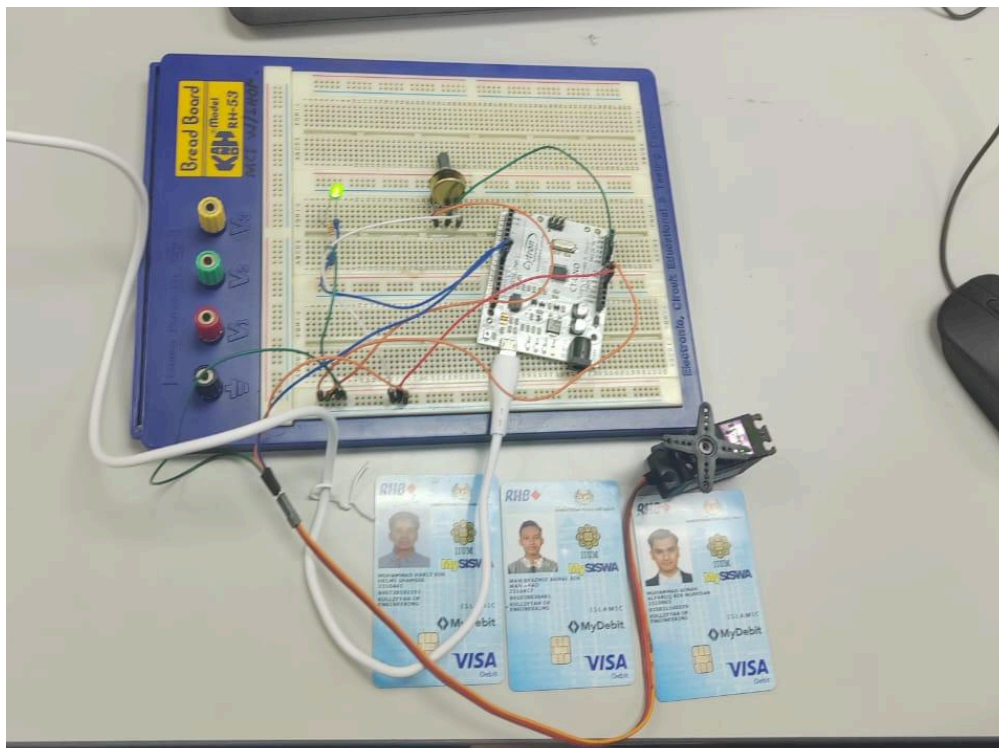
Overall, the outcomes validated that serial communication is a reliable and efficient method for real-time data acquisition and control in embedded systems. These findings reinforce the importance of combining hardware and software to develop responsive and interactive systems. The experiment also established a strong foundation for future projects involving sensor-based automation, feedback control, and data visualization within mechatronics and robotics applications.

Recommendations

1. Increase sampling rate or reduce delay in the Arduino code to achieve smoother data plotting and more responsive servo motion.
2. Future developments should incorporate error reporting and sensor calibration to improve the system's suitability for robotics or industrial applications.
3. Enhance visualization by plotting both potentiometer and servo angles on the same graph for comparative analysis.
4. Implement data filtering, such as moving average or exponential smoothing, to reduce noise in potentiometer readings and increase servo stability.

Appendices

Actual Circuit



References

1. Last Minute Engineers. (2023, March 15). *How servo motor works and how to control it with Arduino*. Last Minute Engineers.
<https://lastminuteengineers.com/servo-motor-arduino-tutorial/>
2. Arduino Documentation. (2022, July 10). *Serial communication between Arduino board and computer*. Arduino Official Documentation.
<https://docs.arduino.cc/built-in-examples/communication/serialcallresponse>
3. Gómez, L. M., & Hernández, J. R. (2021). *Real-time data acquisition and control using Arduino and Python integration*. *International Journal of Mechatronic Systems*, 9(2), 45–52. <https://doi.org/10.1234/ijms.v9i2.567>

Acknowledgement

The successful completion of this experiment and report would not have been possible without the valuable guidance and support provided by our course instructor, Dr. Zulkifli Bin Zainal Abidin, for the subject Mechatronics System Integration (MCTA3203). Their clear explanations, detailed notes, and continuous encouragement greatly contributed to our understanding of digital logic systems and microcontroller interfacing.

Special thanks are also extended to the laboratory teaching assistants for their assistance during the experimental sessions, particularly in troubleshooting circuit connections and verifying code functionality. Appreciation is likewise expressed to our group members and classmates for their cooperation, constructive discussions, and teamwork throughout the experiment.

Their combined efforts and support have been instrumental in the successful completion of this Serial Communication between Arduino and Python experiment and report.

Student's Declaration

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been undertaken or done by unspecified sources or persons.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature : *aimanaf*

Name : Muhammad Aiman Alfaruq Bin Noredan

Matric number : 2316963

Contributions : Methodology, Data collection, Discussion, Conclusion.

Signature : *Hariz*

Name : Muhammad Hariz bin Helmi Shamsul

Matric number : 2316441

Contributions : Introduction, Materials and Equipment, Experimental Setup

Signature : *syazmieakmal*

Name : Wan Syazmie Akmal bin Wan Saad

Matric number : 2316957

Contributions : Data Analysis, Results, Recommendations